adr-0005-delta-updates

ADR-0005 — Delta Updates: AOSP Block Diffs vs Custom (and Phase Placement)

Field	Value
ADR	0005
Title	Delta updates: AOSP block diffs vs custom (and phase placement)
Revision	2
Created	2026-06-07
Last modified	2026-06-07
Status	**Proposed**
Decision owners	Lead architect (synthesis)
Deciders / reviewers	Operator; mandatory code-review subagent (§11.4.125)
Supersedes	—
Superseded by	—
Related	[ADR-0001 engine selection], [ADR-0002 supply-chain trust], [ADR-0004 transport]; `../ota_landscape_report.md`; `../additions_synthesis.md`; `../stacks/aosp-update-engine.md`
Decision needed	Decide the delta-update approach for Helix OTA and which phase it lands in (MVP is likely full-payload-only).
Anti-bluff rule	Every claim traces to a cited source note/report. Unconfirmed facts carried forward as **UNVERIFIED**; this ADR introduces no facts absent from the underlying notes.

> **Fixed (Rev 2):** Corrected Mender Android citation §6→§7; removed unsupported `report §2.3` over-cite from the §1 full-payload "Large" claim (now `aosp-update-engine §5` only); normalized malformed `[rauc §line 99]` tokens to `[rauc line 99]`.

Table of Contents

---

1. Context

Helix OTA is **Android 15 first** (Orange Pi 5 Max / RK3588 class), using **native Android A/B + Virtual A/B** on device with a **custom Go control plane** as the server; Linux/universal is a later phase. [ota_landscape_report §metadata; aosp-update-engine §1] These are **LOCKED** decisions (native A/B + custom Go control plane; research decides the wrap target; mandated stack Go + Gin + Brotli + HTTP/3→HTTP/2, REST-primary, PostgreSQL + MinIO/S3). [ota_landscape_report lines 4–6]

Update artifacts on Android are produced by **`ota_from_target_files`** from the build's `target-files.zip`, yielding `payload.bin` + `payload_properties.txt`. The tool supports two artifact shapes:

- **Full payload** — generated solely from the target image; contains everything needed to write the inactive slot; "Large." [aosp-update-engine §5]
- **Incremental / delta payload** — a differential update produced with the **`-i PREVIOUS-target-files.zip`** flag against the exact source build; "much smaller." [aosp-update-engine §5 lines 107–110]

Two facts shape this ADR. First, **delta generation already exists natively in AOSP** — the same `update_engine` payload generator (`brillo_update_payload` wrapping `delta_generator`) produces both full and incremental `payload.bin`,

and the on-device `applyPayload` path consumes both identically. [aosp-update-engine §5 line 124, §6] Second, **incrementals are tightly version-coupled**: an incremental only applies to its exact source build, gated by `META-INF/com/android/metadata` preconditions (`pre-build`, `pre-build-incremental`, `pre-device`). [aosp-update-engine §5 line 122, §7 item 4]

Independently of artifact format, **Virtual A/B already provides on-device storage/bandwidth savings** via COW compression (`Replace`/gzip, plus XOR diff since Android 13). Reported snapshot-size reductions: ~45% (full OTA) and ~55% (incremental OTA), with XOR adding a further ~25–40% on some devices. These figures are **UNVERIFIED** — sourced from the AOSP Virtual A/B page and flagged version-dependent; confirm against the Android 15/16 docs revision and on the RK3588 board. [aosp-update-engine §4 lines 91–92, §10 item 3]

The operator drafts and synthesis already bias scope toward minimal MVP: the reconciliation defers user-/multi-version rollback past MVP (C6) and treats all unconfirmed delta-related constants as hypotheses for the research ADRs, not settled fact. [additions_synthesis §5 C6; §6]. This ADR is the routed decision point: **"ADR-0005 Delta updates: AOSP block diffs vs custom, and phase placement."** [additions_synthesis §7]

2. Decision Drivers

- **Locked "wrap, do not replace" for the Android device path** — Helix's value-add is build/serve + rollout/telemetry, not re-implementing apply/verify/rollback. A custom delta format would re-derive a hardened, version-tracked AOSP capability for no Android upside. [aosp-update-engine §8 lines 180–182, §9]
- **MVP leanness** — the master design and synthesis favor the smallest correct first version; full-payload-only is the simplest correct artifact and removes the version-matrix and per-pair generation burden from MVP. [additions_synthesis §5 C2, C6]
- **Bandwidth/cost** — full payloads are "Large" [aosp-update-engine §5], so deltas are attractive once a release cadence and a base-build matrix exist; but Virtual A/B compression already recovers significant on-device cost even for full payloads (UNVERIFIED figures). [aosp-update-engine §4]
- **Anti-guessing / research-before-implementation** — concrete savings numbers are UNVERIFIED and must be measured before committing to a delta pipeline (§11.4.6 / §11.4.8). [aosp-update-engine §10; additions_synthesis §6]
- **Verify-before-apply posture** — the synthesis prefers the safer local-verified-file apply over opaque streaming, and keeps SHA-256 + per-artifact signature + the A/B engine's own payload check as defense-in-depth; whichever artifact shape is chosen, the integrity contract is unchanged because TUF/signing treat `payload.bin` as an opaque target. [additions_synthesis §6; ota_landscape_report §3.3 line 198]

3. Options Considered

3.1 Option A — Full payload only (no deltas) for MVP

Serve only full `payload.bin` artifacts produced by `ota_from_target_files` (no `-i`). Every device receives a complete inactive-slot image regardless of its current build. [aosp-update-engine §5]

- **Pros:** Simplest correct artifact; no source-build matrix; one artifact per release; trivially universal across device build states; integrity/serving contract (`ZIP_STORED` + HTTP Range + `FILE_HASH`/`METADATA_HASH`/sizes) is identical to the streaming design already specified. [aosp-update-engine §6 line 147, §7] Virtual A/B COW compression still reduces on-device storage cost for full OTAs (~45%, UNVERIFIED). [aosp-update-engine §4]
- **Cons:** Largest network transfer per update — full payloads are "Large." [aosp-update-engine §5] Bandwidth-sensitive fleets pay full-image cost on every release until deltas land.

3.2 Option B — AOSP-native incremental payloads (`ota_from_target_files -i`)

Use AOSP's built-in incremental mode: `ota_from_target_files -i PREVIOUS-target_files.zip TARGET dist incremental_ota_update.zip`, producing a much smaller `payload.bin` consumed by the same `applyPayload` path. This is the "AOSP block diffs" arm of the decision. [aosp-update-engine §5 lines 107–110, §6]

- **Pros: Native, battle-tested, zero new on-device code** — incrementals are produced by the same generator and applied by the same daemon as full payloads; the device client and integrity contract are unchanged. [aosp-update-engine §5 line 124, §6, §8] Materially smaller transfers ("much smaller" per docs). [aosp-update-engine §5] Pairs with XOR-COW compression for additional on-device savings (~55% incremental, +25–40% XOR; both UNVERIFIED). [aosp-update-engine §4] Stays inside the locked "wrap, do not replace" boundary. [aosp-update-engine §8]
- **Cons: Version coupling** — an incremental applies only to its exact source build, enforced by `pre-build`/`pre-build-incremental` preconditions; the control plane must match each device's reported build to the correct source→target delta. [aosp-update-engine §5 line 122, §7 item 4] Generation cost scales with the number of supported source builds (an N-base → 1-target matrix), and a full-payload fallback is still required for devices off the supported base set. [aosp-update-engine §5, §7 item 4] Exact savings ratios for Helix images are UNVERIFIED until measured. [aosp-update-engine §10]

3.3 Option C — Custom delta format (bespoke block/binary diff)

Design a Helix-specific block/binary diff format and a custom on-device apply path.

- **Rejected.** This re-implements the dangerous, version-tracked apply/verify/rollback surface AOSP already provides, directly contradicting the locked "wrap, do not replace" conclusion for Android: "Reproducing them outside AOSP would mean re-deriving years of hardened boot/verify/rollback logic and tracking it across Android releases — a maintenance liability with no upside on

Android." [aosp-update-engine §8 lines 180–182] A custom format would also break the clean integrity seam where TUF/signing treat `payload.bin` as an opaque target. [ota_landscape_report §3.3 line 198] No source note presents any Android benefit from a bespoke format over AOSP-native incrementals.

3.4 Option D — Wrap a third-party delta generator

Adopt an external OTA platform's delta engine (e.g., Mender server-side delta generation).

- **Rejected for the Android path.** Mender has **no Android client and no `update_engine` integration**; its A/B is bootloader-driven (U-Boot/GRUB), fundamentally different from Android's `update_engine` + boot_control HAL + Virtual A/B. [mender §7 line 88; ota_landscape_report §4] Mender's delta generation (auto/server-side) is a **paid Professional/Enterprise feature**, and its ~70–90% bandwidth-savings figure is a **vendor claim, UNVERIFIED** as an independent benchmark — conflicting with both the open-control-plane differentiator and the no-bluff rule. [mender §5 line 69, §9 lines 111–112, line 140; ota_landscape_report §4] hawkBit, the front-runner wrap candidate, treats artifacts as **opaque** and provides no delta generation of its own. [ota_landscape_report §2.3 line 96 (A/B=1, artifacts opaque)]

3.5 Linux-phase deltas (out of scope for the Android decision)

For the later Linux/universal phase, bandwidth-efficient deltas exist in the shortlisted Linux engines and should be evaluated then, not now:

- **OSTree static deltas** — pre-computed server-generated commit-to-commit diffs (`ostree static-delta generate`), using bsdiff for similar files plus fallback objects; "bandwidth-efficient updates without a custom diff format." Concrete size ratios are **UNVERIFIED** pending benchmarking on representative images. [ostree §5 lines 83–93, line 158]
- **RAUC adaptive/casync** — RAUC distinguishes *adaptive* updates (`block-hash-index`, skip unchanged blocks) from *delta* (casync chunked diff needing a separate chunk store); enabling streaming disables plain casync bundles. [rauc line 99]

These belong to the Linux OS-adapter and are deferred to the Linux-phase engine ADR, consistent with the report's "shortlist for the later Linux phase" verdicts. [ota_landscape_report §4]

4. Decision

MVP (1.0.0): full payload only (Option A). Ship complete `payload.bin` artifacts; do not generate or serve incrementals in MVP. Rely on **Virtual A/B COW compression for on-device savings** in the interim. [aosp-update-engine §4, §5]

Post-MVP (1.0.1+): adopt AOSP-native incremental payloads (Option B); reject custom and third-party delta formats (Options C, D). When deltas are introduced, use `ota_from_target_files -i` to produce source→target

incrementals applied by the unchanged `applyPayload` path, with **full payloads retained as the universal fallback** for devices off the supported base-build set. [aosp-update-engine §5 lines 107–110, §7 item 4]

This honors the LOCKED decisions: native A/B + custom Go control plane (the control plane owns base-build matching and serving; AOSP owns apply); research decides the wrap target (no wrap dependency is taken for deltas — the native AOSP generator is used directly); and the mandated stack (deltas are served over the same `ZIP_STORED` + HTTP Range + Brotli + HTTP/3→HTTP/2 path with no new device code). [aosp-update-engine §6–§9; ota_landscape_report lines 4–6]

The introduction of incrementals is **gated on measured savings** on representative Helix images / the RK3588 board (anti-guessing, §11.4.6 / §11.4.8); the UNVERIFIED COW/XOR percentages must not be quoted as fact in any contract or commitment. [aosp-update-engine §4, §10]

5. Consequences

5.1 Positive

- **MVP is the simplest correct artifact path** — one full payload per release, no source-build matrix, universal across device build states, identical integrity/serving contract to the existing streaming design. [aosp-update-engine §5, §6, §7]
- **No on-device delta code, ever** — both full and (later) incremental payloads flow through the same `applyPayload`/callbacks surface; the thin Android client is unaffected by the delta decision. [aosp-update-engine §6, §8]
- **Clean integrity seam preserved** — `payload.bin` stays an opaque, signed target file for SHA-256/signature and (later) TUF, regardless of full-vs-incremental shape. [ota_landscape_report §3.3 line 198; additions_synthesis §5 C5]
- **Clear, low-risk upgrade path** — moving to native incrementals later is additive (a build-pipeline + control-plane-targeting change), not a device or format change. [aosp-update-engine §5, §7]

5.2 Negative / costs

- **MVP pays full-image bandwidth on every release** — full payloads are "Large"; bandwidth-sensitive fleets bear higher transfer cost until 1.0.1+ deltas land. [aosp-update-engine §5]
- **Deferred complexity, not eliminated** — incrementals introduce a **source→target base-build matrix**, per-pair generation cost, precondition matching in the control plane, and a mandatory full-payload fallback. [aosp-update-engine §5, §7 item 4]
- **Savings are UNVERIFIED** — the headline COW/XOR and incremental-size figures are unconfirmed and version/board-dependent; the business case for deltas cannot be quoted until measured. [aosp-update-engine §4, §10]

5.3 Neutral / follow-ups

- Telemetry must already model Virtual A/B states ("applied, pending reboot, merging, merged"); this is independent of the delta decision but interacts with measuring real-world apply/merge cost. [aosp-update-engine §4 line 94, §9 line 194]
- Linux-phase deltas (OSTree static deltas / RAUC adaptive) are deferred to the Linux engine ADR. [ostree §5; rauc line 99; ota_landscape_report §4]

6. Phase Placement

Phase	Delta posture	Rationale (traceable)
1.0.0-MVP	**Full payload only.** Virtual A/B COW compression provides interim on-device savings.	Simplest correct artifact; MVP leanness; savings figures UNVERIFIED. [aosp-update-engine §4, §5; additions_synthesis §5 C2/C6]
1.0.1+	**AOSP-native incrementals (`-i`)** with full-payload fallback; gated on measured savings.	Native, no new device code, smaller transfers; version-coupling/matrix cost justified once a release cadence exists. [aosp-update-engine §5, §7]
Linux phase	Evaluate **OSTree static deltas** / **RAUC adaptive** in the Linux OS-adapter ADR.	Linux engines shortlisted for the later phase only. [ota_landscape_report §4; ostree §5; rauc line 99]

This placement matches the synthesis's MVP-minimal bias and routes the delta question exactly as the open-questions list intended. [additions_synthesis §5 C2/C6, §7]

7. Open / UNVERIFIED Items to Close Before Implementation

1. **Virtual A/B COW/XOR savings** (~45% full / ~55% incremental / +25–40% XOR) — **UNVERIFIED**; confirm against the Android 15/16 docs revision and measure on RK3588 / Orange Pi 5 Max. [aosp-update-engine §4 lines 91–92, §10 item 3]
2. **Real incremental-vs-full size ratio on Helix images** — not yet measured; the 1.0.1+ delta business case depends on it. [aosp-update-engine §5, §10]
3. **Exact `ota_from_target_files` incremental + signing flag spelling** for the Android 15 docs revision (`-i`, `--package_key`, AVB/`vbmeta`). **UNVERIFIED.** [aosp-update-engine §5 line 117, §10 item 4]
4. **Precondition/targeting fields** (`pre-build`, `pre-build-incremental`, `pre-device`) the control plane must match for safe incremental delivery, plus

optional headers (`SWITCH_SLOT_ON_REBOOT`, `RUN_POST_INSTALL`, `DISABLE_DOWNLOAD_RESUME`). **UNVERIFIED** exact set. [aosp-update-engine §5 line 122, §7 item 3, §10 item 1]
5. **Board confirmation** — whether the RK3588 target build ships classic A/B or Virtual A/B affects realized COW savings; needs hands-on confirmation. [aosp-update-engine §10 item 6]
6. **Mender delta savings** (~70–90%) remain a **vendor claim, UNVERIFIED** — do not cite if Option D is ever revisited. [mender line 140]

8. Status

Proposed. Pending operator review gate and the mandatory code-review subagent pass (§11.4.125). No implementation of deltas is authorized until the gating measurements in §7 are closed (§11.4.6 / §11.4.8).

9. Compliance Notes (HelixConstitution)

> Clause meanings sourced from the synthesis and master design, which cite the HelixConstitution; the full Constitution is not co-located in this repo, so clause numbers are carried forward exactly as those canonical docs use them. [additions_synthesis §1 line 34; 2026-06-07-helix-ota-design §15 lines 183–189]

Clause	How this ADR complies
§11.4.6 (no-guessing)	Every savings/version constant is marked **UNVERIFIED** and excluded from any commitment; the delta business case is explicitly gated on measurement, not estimated. [additions_synthesis §1 line 34, §6]
§11.4.8 (research-before-implementation)	This is an evidence-based ADR routed from `additions_synthesis §7`; incremental adoption is deferred until §7 spikes/measurements are closed. [additions_synthesis §7; design §2 D3]
§7.1 (no-bluff / positive evidence)	Vendor marketing (Mender's 70–90%, draft "most comprehensive" framing) is rejected, not propagated; every claim cites a source note. [additions_synthesis §6; design §15 line 184]
§11.4.74 (catalogue-first reuse)	Reuses the native AOSP delta generator and existing serving/transport submodules (`http3`, `Storage`/MinIO) rather than building a bespoke format; Custom (C) and third-party (D) rejected. [aosp-update-engine §5, §6; design §10 line 139]
§11.4.28 (decoupling)	The delta decision changes only build-pipeline + control-plane targeting; the `update_engine` bridge and OS-adapter

Clause	How this ADR complies
	seams are untouched, so the Linux phase can choose its own delta mechanism independently. [design line 107; ostree §5]
§1 / §1.1 (four-layer testing + mutation immunity)	Delta apply is a safety-critical path (apply + signing-verify): targets ≥90% coverage, with emulated A/B apply and a real-board download→verify→apply→reboot→verify + corrupt-slot fallback plan. [design §13 lines 173–175]
§11.4.125 (mandatory code-review gate)	ADR stays **Proposed** until the adversarial code-review subagent and operator review gate pass. [design §14 line 177]

10. Sources

All paths relative to `docs/research/main_specs/`:

- `research/ota_landscape_report.md` — landscape report & engine-selection synthesis (LOCKED strategy, hawkBit/Mender verdicts, opaque-artifact note).
- `research/additions_synthesis.md` — operator-draft reconciliation (C2/C5/C6 resolutions; ADR-0005 routing; corrections of guessed/vendor claims; clause meanings).
- `research/stacks/aosp-update-engine.md` — AOSP `update_engine` / native A/B + Virtual A/B; `ota_from_target_files` full vs incremental (`-i`); `applyPayload` contract; COW/XOR figures (UNVERIFIED); preconditions/targeting.
- `research/stacks/mender.md` — Mender delta generation (paid; vendor savings claim UNVERIFIED); no Android client.
- `research/stacks/ostree.md` — OSTree static deltas (Linux phase).
- `research/stacks/rauc.md` — RAUC adaptive vs delta (Linux phase).
- `00-master/2026-06-07-helix-ota-design.md` — master design (D3 engine-choice deferral, constitution clause mapping, testing/decoupling clauses).

> No statistics, dates, or citations were fabricated. Items not confirmed against a primary source are tagged **UNVERIFIED**.